

Interface Inteligente para Consulta de Arquivos CSV

Relatório Técnico -- Desafio 4

I2A2 -- Instituto de Inteligência Artificial Aplicada

Agentes Inteligentes e LLMs

Framework de agentes: LangChain (create_agent / LangGraph)

Modelo de linguagem: Groq -- openai/gpt-oss-20b

Interface: Streamlit (3 abas)

Integrantes		
Isabel de Castro Beneyto	(85) 98630-5456	castrobeneyto@gmail.com
Adolfo Emmanuel Correa López	(21) 97240-9801	adolfo.correa.lopez@gmail.com
Jessica Mayumi Odo Bastos	(11) 95216-6175	jessica.odo03@gmail.com
Diogo David Macêdo Sena	(84) 99982-4141	diogodmsena@gmail.com

1. Introdução e objetivo pedagógico

O objetivo deste projeto é aplicar, na prática, os conceitos do curso sobre agentes inteligentes, LLMs, ferramentas (tools) e orquestração, construindo uma solução capaz de transformar dados estruturados (CSV) em informação útil a partir de perguntas em linguagem natural.

Diferente de um chatbot genérico, que apenas gera texto probabilístico (sujeito a "chutar" números), esta solução implementa o padrão **AI Agent com Tool Calling**: a LLM nunca calcula nada de cabeça -- ela decide qual ferramenta chamar e essa ferramenta executa código real (pandas/plotly) sobre os dados carregados. Três ideias guiaram as escolhas de design:

- Determinismo numérico -- toda resposta quantitativa vem da execução real de uma tool, nunca de memória da LLM.
- Transparência (Explainable AI) -- cada decisão do agente é exposta ao usuário (trace de tools chamadas, código executado, resultado real).
- Multimodalidade -- a resposta combina texto executivo, tabela de dados e gráfico, conforme apropriado à pergunta.

2. Framework escolhido

O agente foi construído com **LangChain**, usando a API atual de agentes da versão 1.x (`langchain.agents.create_agent`), que orquestra o loop de tool-calling internamente via LangGraph. O modelo de linguagem é o **openai/gpt-oss-20b**, servido pela **Groq** (via `langchain-groq`). A interface foi feita em **Streamlit**, o que também permite publicação gratuita no Streamlit Community Cloud.

Por que Groq e não uma LLM proprietária: testamos primeiro o Google Gemini (`gemini-2.5-flash`) e depois o Groq com `llama-3.3-70b-versatile` -- ambos com tier gratuito, mas com cotas diárias baixas demais para uso real em sala de aula: o Gemini limita 20 requisições/dia por projeto, e o Llama 70B na Groq estourava sua cota de *tokens*/dia (100k) em poucas perguntas. Testamos então um modelo menor (`llama-3.1-8b-instant`), mas as respostas finais perderam os números concretos exigidos pelo prompt. O `openai/gpt-oss-20b` na Groq manteve a qualidade das respostas equivalente à do modelo 70B, com uma cota de *tokens*/dia bem mais folgada -- adequado para múltiplos integrantes do grupo testarem sem custo.

3. Pilha de tecnologias

Bibliotecas efetivamente declaradas em `pyproject.toml` / `uv.lock`, e o papel de cada uma:

Biblioteca	Versão	Papel no projeto
Streamlit	<code>>=1.38.0</code>	Interface web reativa (abas, formulários, chat, gráficos) em Python puro.
LangChain	<code>>=0.3.0</code>	Orquestração do agente (<code>create_agent</code>), histórico de conversação e integração de tools.
<code>langchain-groq</code>	<code>>=1.1.3</code>	Cliente LangChain para a Groq -- LLM de baixíssima latência (hardware LPU dedicado), tier gratuito viável para uso em grupo.

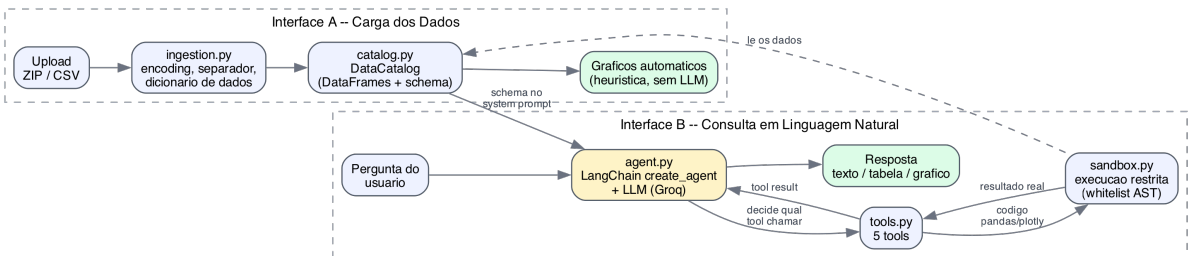
LangGraph	<code>>=0.2.0</code>	Motor de execução por trás do <code>create_agent</code> -- loop de tool-calling como grafo de estados.
Pandas	<code>>=2.2.0</code>	Motor analítico: agregações, groupby, ordenação, datas -- é o código que a LLM escreve e a sandbox executa.
Plotly	<code>>=5.24.0</code>	Gráficos interativos (zoom, hover) renderizados no navegador via WebAssembly.
chardet	<code>>=5.2.0</code>	Detecção heurística de encoding -- essencial para CSVs brasileiros fora de UTF-8 (ex: latin-1).
python-dotenv	<code>>=1.0.1</code>	Carrega a chave de API de um <code>.env</code> local, mantendo segredos fora do código-fonte.
tabulate	<code>>=0.9.0</code>	Formata resultados de pandas como markdown legível no trace do agente.
ast (nativo)	<code>builtin</code>	Valida a árvore sintática do código gerado pela LLM antes de executá-lo (sandbox.py).
Graphviz	<code>via st.graphviz_chart</code>	Renderiza o diagrama de arquitetura a partir de uma descrição textual (DOT) -- sem dependência de pacote extra.
uv	ferramenta de projeto	Gerenciador de dependências e ambiente (Rust) -- reprodutibilidade exata via <code>uv.lock</code> .

4. Arquitetura da solução

A aplicação é dividida em módulos de responsabilidade única, sem lógica de negócio misturada na interface:

Arquivo	Responsabilidade
app.py	Interface Streamlit: Aba A (upload + datasets de exemplo), Aba B (chat), Aba C (arquitetura)
src/csvagent/ingestion.py	Extraí o ZIP; detecta encoding e separador por arquivo; identifica o dicionário de dados; normaliza colunas com inteiros gigantes (ex: chave de acesso de NF-e) para evitar OverflowError do pyarrow
src/csvagent/catalog.py	Catálogo em memória: DataFrames + schema + dicionário de dados; gera o resumo textual para o prompt
src/csvagent/autocharts.py	Gráficos genéricos por heurística sobre o schema (sem LLM): tanto ao carregar os dados quanto sobre os resultados de uma consulta; sugestões de pergunta baseadas nas colunas reais
src/csvagent/sandbox.py	Execução restrita (whitelist de AST) do código pandas/plotly gerado pelo agente
src/csvagent/tools.py	5 tools do agente + registro do trace de decisão para a UI
src/csvagent/agent.py	Monta o agente: system prompt + LLM (Grok) + tools

Diagrama do fluxo completo



Módulos em verde não dependem da LLM (heurística pura); o módulo em amarelo (agent.py) é onde o LangChain + Groq efetivamente decidem o que fazer. O mesmo diagrama é exibido ao vivo na Aba 3 ("Arquitetura") do app.

Interface A -- Carga dos dados

O usuário envia um .zip com um ou mais CSVs (ou CSVs soltos) e, opcionalmente, um dicionário de dados -- ou usa um dos **datasets de exemplo integrados** (notas fiscais de serviço, NF-e de equipamentos, base de compras), carregados com um clique para facilitar a demonstração. O pipeline de ingestão detecta encoding e separador automaticamente, classifica cada arquivo como dado ou dicionário, e sugere até 2 gráficos automáticos por tabela via heurística de schema (sem custo de LLM).

Interface B -- Consulta em linguagem natural

O catálogo (schema + amostra + dicionário) é injetado no system prompt do agente. A cada pergunta, o agente decide livremente qual sequência de tools chamar; a resposta final é acompanhada de tabela, gráfico (quando gerado) e do trace completo de decisão.

Interface C -- Arquitetura

Terceira aba, que mostra o mesmo diagrama Graphviz reproduzido acima, ao vivo -- torna a arquitetura auto-explicativa dentro do próprio app.

Sandbox de execução

O código pandas/plotly escrito pelo LLM não é executado livremente: passa antes por um validador de AST (`sandbox.py`) que bloqueia `import/__import__` (acesso a módulos como `os`, `sys`, `subprocess`), `eval/exec/compile/open`, atributos `__dunder__` (`__class__`, `__subclasses__`, `__globals__`), definição de função/classe e loops `while`. Os únicos nomes expostos ao código são `pd`, `px/go` e o dicionário `tables` com cópias dos DataFrames carregados. Não é isolamento de processo/SO -- é uma defesa em profundidade adequada ao escopo educacional do desafio.

5. Descrição do agente e das ferramentas (tools)

A solução usa um único agente (arquitetura enxuta, conforme recomendado pelo enunciado), equipado com 5 tools especializadas:

- **listar_tabelas** -- Lista as tabelas carregadas, com número de linhas e colunas.
- **descrever_tabela** -- Mostra schema detalhado (nome/tipo de cada coluna + descrição do dicionário de dados), estatísticas descritivas e uma amostra de linhas.
- **valores_frequentes** -- Retorna os valores mais frequentes de uma coluna categórica, com contagem.
- **executar_pandas** -- Executa uma consulta pandas real sobre as tabelas e retorna o resultado real, nunca um número "chutado".
- **gerar_grafico** -- Gera um gráfico Plotly (px/go) a partir das tabelas, quando útil para a resposta.

O system prompt instrui explicitamente o modelo a nunca responder de memória, e a nunca inventar tags de imagem markdown (uma alucinação observada nos testes com este modelo, corrigida via instrução explícita).

6. Jornada didática do agente ("Bastidores do Agente")

Para tornar o raciocínio do agente auditável, a interface decompõe cada resposta em 5 etapas sequenciais, exibidas ao usuário:

- 1 Recepção e interpretação da pergunta -- extrai a intenção analítica e os parâmetros temporais/categóricos da pergunta em linguagem natural.
- 2 Inspeção do dicionário de dados e das tabelas -- o agente confirma nomes exatos de colunas e consulta o dicionário de dados antes de calcular qualquer coisa.
- 3 Planejamento e geração de código pandas -- traduz a intenção em uma consulta estruturada (agrupamento, soma, filtro).
- 4 Execução em sandbox -- validação por AST e execução restrita; o resultado real (dimensão da tabela, tempo de resposta) volta para o agente.
- 5 Síntese e formatação -- resposta executiva em texto, mais tabela e gráfico exibidos automaticamente.

7. Fluxo de funcionamento da aplicação

- 1 Usuário envia um .zip/CSV(s) na Aba A, ou carrega um dataset de exemplo integrado com um clique.
- 2 ingestion.py extrai, detecta encoding/separador, classifica cada arquivo (dado vs. dicionário) e normaliza colunas problemáticas.

- 3 `catalog.py` monta o catálogo em memória e o resumo textual do schema; `autocharts.py` sugere até 2 gráficos genéricos e perguntas de exemplo baseadas no schema real.
- 4 Ao abrir a Aba B, `agent.py` monta o agente: system prompt (catálogo) + LLM (Groq) + as 5 tools.
- 5 Usuário faz uma pergunta em linguagem natural no chat (ou clica numa sugestão).
- 6 O agente (LangGraph por baixo) decide, em loop, quais tools chamar -- cada chamada roda de fato sobre os dados (via `sandbox.py`) e o resultado real volta para o agente.
- 7 A resposta final é apresentada de forma multimodal: resumo executivo, tabela e gráfico, além do trace completo em 5 etapas ("Bastidores do Agente").
- 8 A Aba C exibe, a qualquer momento, o diagrama da arquitetura completa.

8. Perguntas realizadas e respostas obtidas

Testes realizados com um CSV sintético de notas fiscais (5 linhas: id_nf, fornecedor, produto, categoria, valor, data_emissao) acompanhado de um dicionário de dados no mesmo .zip, processado end-to-end pela aplicação real (upload -> ingestão -> agente -> resposta) com o modelo de produção (Groq openai/gpt-oss-20b). Para cada pergunta, o trace real do agente (tool chamada + código executado) é mostrado abaixo do texto.

P1. Qual fornecedor recebeu o maior valor no período?

Resposta do agente: O fornecedor que recebeu o maior valor no período foi Fornecedor A, com um total de R\$ 601,25.

tool: executar_pandas

```
resultado = tables["notas"].groupby("fornecedor")["valor"].sum().sort_values(ascending=False).head(1)
```

P2. Qual produto apresentou o maior volume comprado (em quantidade de notas)?

Resposta do agente: O produto que apareceu em mais notas fiscais é Parafuso, com 2 notas registradas.

tool: executar_pandas

```
resultado = tables["notas"].groupby("produto").size().reset_index(name="count").sort_values("count", ascending=False).head(1)
```

P3. Qual foi o total gasto em cada mês?

Resposta do agente: Total gasto por mês: 2024-01: R\$ 350,50 -- 2024-02: R\$ 651,00 -- 2024-03: R\$ 80,00. (o agente também gerou um gráfico de barras automaticamente, por iniciativa própria)

tool: executar_pandas

```
resultado = tables["notas"].assign(mes=pd.to_datetime(tables["notas"].data_emissao).dt.to_period('M')).groupby("mes")["valor"].sum().reset_index()
```

tool: gerar_grafico

```
fig = px.bar(resultado, x='mes', y='valor', labels={'mes': 'Mês', 'valor': 'Total Gasto (R$)'}, title='Total gasto por mês')
```

P4. Quais foram os cinco maiores fornecedores?

Resposta do agente: Fornecedor A (R\$ 601,25), Fornecedor B (R\$ 330,00), Fornecedor C (R\$ 150,25). O agente observou corretamente que o dataset só tem 3 fornecedores, então não há 5 para listar.

tool: executar_pandas

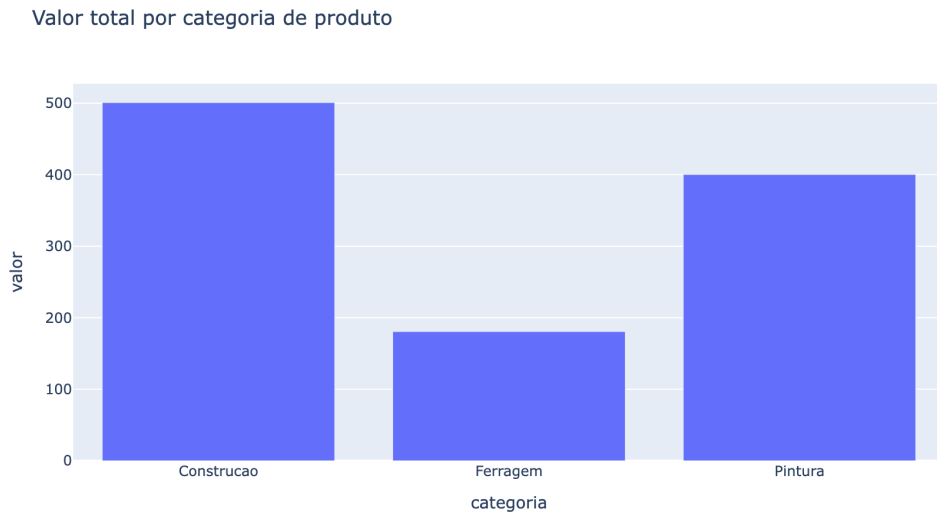
```
resultado = tables["notas"].groupby("fornecedor")["valor"].sum().sort_values(ascending=False).head(5)
```

P5. Gere um gráfico do valor total por categoria de produto.

Resposta do agente: "Gráfico gerado com sucesso e será exibido para o usuário."

tool: gerar_grafico

```
fig = px.bar(tables["notas"].groupby("categoria")["valor"].sum().reset_index(),
             x="categoria", y="valor", title="Valor total por categoria de produto")
```



9. Limitações conhecidas (MVP)

- Um único dicionário de dados é aplicado a todas as tabelas do ZIP; não há suporte a um dicionário por tabela.
- O sandbox impede `while` e algumas funções perigosas, mas não há timeout de execução -- um `for` sobre uma tabela muito grande ainda pode ser lento.
- Sem suporte a múltiplos ZIPs na mesma sessão (recarregar substitui o catálogo carregado).
- O tier gratuito da Groq tem cota diária de tokens (bem mais folgada que a de LLMs proprietárias, mas ainda finita) -- para uso contínuo com muitos integrantes, recomenda-se cada um usar sua própria chave.
- Os datasets de exemplo integrados (sample_data/*.zip) são ignorados pelo .gitignore (*.zip) -- precisam ser adicionados manualmente ao ambiente de deploy para os botões de carga rápida funcionarem fora da máquina onde foram criados.
- Os gráficos e perguntas automáticos são heurísticos (baseados em tipo de coluna e cardinalidade) -- servem como visão geral inicial, não substituem uma análise dirigida.

10. Como executar

```
git clone https://github.com/castroisabel/i2a2.git
cd i2a2/desafio-4-csv-agent
uv sync
# criar .env com GROQ_API_KEY=... (ou colar a chave na barra lateral do app)
uv run streamlit run app.py
```

Roteiro de demonstração

- 1 Aba 1 (Carga dos Dados): carregue um dataset de exemplo ou envie o seu; observe as perguntas e os gráficos sugeridos automaticamente a partir do schema real.
- 2 Aba 2 (Consulta): clique numa pergunta sugerida ou digite a sua.
- 3 Explore a resposta multimodal: resumo executivo, tabela, gráfico, e o trace em 5 etapas ("Bastidores do Agente") com o código Python realmente executado.
- 4 Aba 3 (Arquitetura): apresente o diagrama e os módulos da solução.

11. Conclusão e lições aprendidas

A implementação reforça alguns aprendizados centrais sobre agentes de IA aplicados a dados tabulares:

- Agentes que agem via código ("code-as-action") são muito mais confiáveis do que respostas puramente textuais para perguntas quantitativas -- o número vem de uma execução real, não de uma estimativa da LLM.
- Segurança em primeiro lugar: qualquer ambiente que execute código gerado por LLM precisa de defesas em camadas (validação de AST, escopo restrito de variáveis, sem I/O).

- A escolha do provedor de LLM tem que considerar cota de uso, não só qualidade -- descobrimos isso na prática ao esgotar o tier gratuito do Gemini e depois o de um modelo grande na Groq, antes de chegar num equilíbrio de custo x qualidade.
- Design e transparência importam: expor o raciocínio do agente (trace de tools, código executado) constrói confiança e facilita o diagnóstico tanto para o usuário final quanto para quem está aprendendo a construir esse tipo de sistema.